

Fundamentos de Telecomunicações

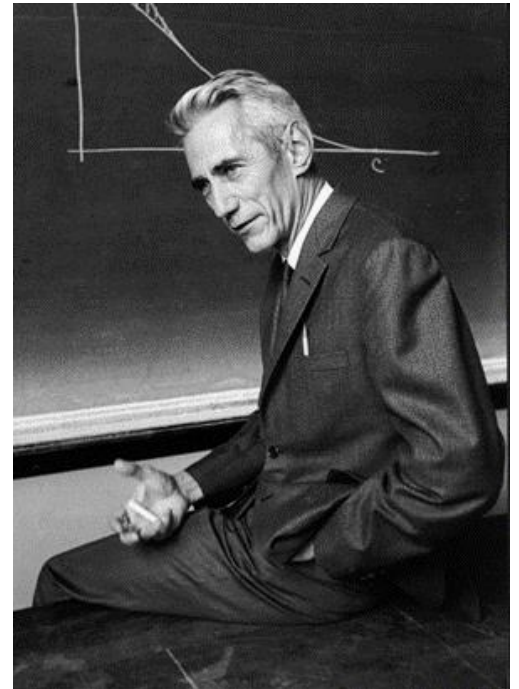
5ª aula

Fontes de Informação
Source Coding



Introdução

- Codificação de Fonte (*Source Coding*) e Codificação de Canal (*Channel Coding*) são conceitos que pertencem à origem das Comunicações Digitais.
- Estas ideias surgiram de um artigo publicado por *Claude Shannon* com o título:
 - “*A mathematical theory of Communication*”



Introdução

- Uma das vantagens principais da comunicação digital é que, para a comunicação acontecer com êxito, não é importante transmitir as mensagens em si. É suficiente transmitir apenas uma representação sem perdas da fonte
 - Por exemplo, se a fonte tem apenas 2 possíveis mensagens (por exemplo, "SIM" ou "NÃO") para transmitir, então é suficiente simplesmente transmitir um 0 (mapeado para "SIM", por exemplo) e um 1 (para "NÃO")
 - O recetor pode facilmente reconstruir a mensagem real da fonte desde que saiba esse mapeamento: {YES, NO} e {0,1}
- Então *Shannon* fez a seguinte pergunta?
 - "Dado um canal sem ruído, qual é a forma mais eficiente de comunicar a mensagem da origem para o destino?"



Introdução

- A ideia base é transmitir os símbolos mais prováveis com códigos com comprimento mais curto (menos bits)
- O objectivo da codificação de fonte é:
 - Transmitir com menos bits o mais possível de informação de forma a aumentar a eficiência de transmissão
- O problema é que os canais de comunicação não estão livres de ruído!
 - Ruído significa essencialmente aleatoriedade ou imprevisibilidade.
- *Shannon*, criou um modelo probabilístico para o canal de forma a obter os valores de determinados símbolos de saída em função de determinados símbolos de entrada



Introdução

- Nesta altura, *Shannon* fez um nova pergunta:
 - Qual é a taxa máxima a que podemos transmitir através de um canal com baixa probabilidade de erro?
- Nasceu assim a Codificação de Canal (*Channel Coding*)
 - Projetar o mapeamento dos símbolos de informação (com código fonte) de forma a atravessar um canal de comunicação com ruído, à taxa máxima possível, com baixa probabilidade de erro
 - A técnica para ser possível controlar a ocorrência de erros, num canal de comunicação, consiste em adicionar redundância na emissão de forma a poder ser possível, através de uma dada técnica, conseguir detetar ou corrigir os bits em erro, na receção
 - *Error Control Coding*



Codificação de Fonte

- Uma fonte de informação é um processo estocástico gerador de mensagens sucessivas
 - Representado por uma função mensurável que varia aleatoriamente no tempo e no espaço de probabilidade
- Numa fonte sem memória, cada mensagem é uma variável aleatória independente e identicamente distribuída
 - Caracterizada pelas propriedades de ergodicidade e estacionariedade do processo gerador
- Processo estocástico (definição):
 - Família de variáveis aleatórias que representam a evolução de um sistema de valores com o tempo.
 - É o contrário de um processo determinístico que possui um único modo de evoluir. Num processo estocástico, existem várias, por vezes infinitas, direções nas quais o processo pode evoluir



Estacionariedade e Ergodicidade

- Estacionariedade:
 - Um processo é estacionário se todas as propriedades estatísticas forem invariantes para uma translação do tempo
- Ergodicidade
 - Um processo $\mathbf{x}(n)$ é ergódico se todas as suas propriedades estatísticas se poderem determinar a partir de uma única realização.
 - É possível substituir médias de conjuntos por médias temporais

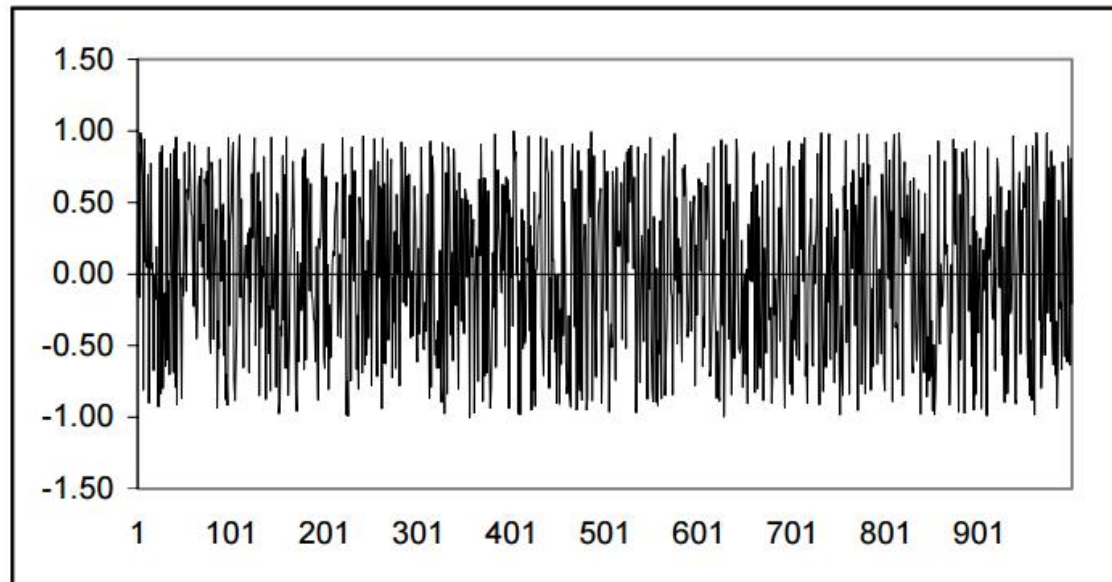
$$E[g(n)] = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N g(n)$$

- Apesar destas propriedades estarem associadas, estacionariedade não implica ergodicidade nem ergodicidade implica estacionariedade



Estacionariedade e Ergodicidade

- Exemplo:
 - Números aleatórios com distribuição uniforme em $[-1, 1[$



- Trata-se de um processo estacionário e ergódico de média nula

Codificação de Fonte

- Alfabeto Binário
 - Um código binário estabelece o mapeamento das mensagens da fonte, traduzindo os símbolos do alfabeto da fonte, em palavras compostas por símbolos do alfabeto binário $\{0,1\}$
 - Cada mensagem da fonte pode ser constituída por um ou mais símbolos do alfabeto da fonte, ou seja, pode ter um comprimento fixo ou variável
 - Exemplo: Código ASCII
 - Estabelece o mapeamento de cada um dos seus 64 ou 256 símbolos (caracteres) do alfabeto da fonte em palavras do alfabeto do código binário, compostas por 6 ou 8 símbolos binários (bits)



Codificação de Fonte

- Comprimento do Código

- Num código de comprimento variável, com um espaço amostral S_x , os símbolos do alfabeto do código apresentam um comprimento $l(x)$
- O comprimento médio do código $\bar{l}$ traduz o valor médio do símbolo no alfabeto, ponderado pela probabilidade de ocorrência $P(x)$ da respectiva mensagem da fonte de informação:

$$\bar{l} = \sum_{S_x} P(x) l(x)$$

- Quantidade de Informação:

- A definição de conteúdo informativo de uma fonte, para a qual se toma como modelo uma variável aleatória, deve depender, naturalmente, das probabilidades dos respectivos símbolos
- Quanto maior for a incerteza associada a uma fonte, maior é a quantidade de informação que é transmitida a um observador por cada amostra gerada por essa fonte



Codificação de Fonte

- Quantidade de Informação (continuação):

- Assim, a medida de informação pode ser vista como uma medida de incerteza – Entropia

$$H(X) = - \sum_{s_x} P(x) \log P(x)$$

- Depende apenas das probabilidades dos símbolos da fonte
- Para duas fontes X e Y , independentes, a incerteza (Entropia) associada, que se escreve $H(X, Y)$, deve ser a soma das incertezas, isto é:
 - Se X e Y independentes $\Rightarrow H(X, Y) = H(X) + H(Y)$
- Um código diz-se ótimo se o seu comprimento for igual à entropia da fonte de informação: $l = H(x)$
- Resulta do facto do comprimento médio dum código l não poder ser inferior à entropia da respectiva fonte de informação X :

$$l \geq H(X) = - \sum_{s_x} P(x) \log P(x)$$



Código

- Um código é uma forma de representar os símbolos de uma dada fonte
- Atribui-se a cada símbolo gerado pela fonte uma sequência de símbolos do alfabeto sobre o qual está definido o código
- Considere-se uma fonte sem memória, discreta, que gera símbolos de um alfabeto
 - $X = \{x_1, \dots, x_N\}$
- Um codificador, definido sobre o alfabeto D , é uma função:
 - $C : X \rightarrow D^*$
 - D^* representa o conjunto de todas as sequências finitas de símbolos de D
 - D^k representa a k -ésima a potência cartesiana do conjunto D , isto é, o conjunto de todas as sequências de k elementos de D



Código

- Exemplo (caso binário):
 - No caso binário, $D = \{0, 1\}$, com $k = 3$, tem-se D^3
 - $D^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$
 - A notação D^* representa o conjunto (infinito) de todas as sequências finitas de elementos de D
 - No caso binário, $D = \{0, 1\}$,
 - $D^* = \{0, 1, 00, 01, 10, 11, 000, 001, \dots, 111, 0000, 0001, 0010, \dots\}$.
 - Assim, um código atribui a cada símbolo do alfabeto da fonte X , uma sequência finita de símbolos de D
- Exemplo:
 - Considere-se o alfabeto de fonte $X = \{a, b, c, d, e\}$ e o alfabeto de código $D = \{0, 1, 2\}$;
 - um exemplo de código para X , definido sobre D é:
 - $C(a) = 0, C(b) = 10, C(c) = 22, C(d) = 2, C(e) = 210$



Código

- Exercício:
 - Para o código definido no exemplo anterior, determine o comprimento médio do código, sabendo que as probabilidades de ocorrência dos símbolos são:
 - $p(a) = 0.4, p(b) = 0.2, p(c) = 0.15, p(d) = 0.15, p(e) = 0.1$
- Resolução:
 - $l(a) = 1, l(b) = 2, l(c) = 2, l(d) = 1, l(e) = 3$
 - $l = 0.4 \times 1 + 0.2 \times 2 + 0.15 \times 2 + 0.15 \times 1 + 0.1 \times 3 = 1.55$
 - Exprime-se em unidades “símbolos de D por símbolo de X ”
 - No caso binário: bit/símbolo



Código Não Singular

- Código Não Singular:
 - A condição mais fraca a impor a um código para que este seja utilizável é que este seja “não singular”, isto é, que a função:
 - $C : X \rightarrow D^*$
 - seja injetiva:
 - $x_1 \neq x_2 \Rightarrow C(x_1) \neq C(x_2)$
 - x_1 e x_2 - são 2 símbolos arbitrários de X
 - Esta condição garante que se se enviar uma palavra de código para um recetor, este pode descodificá-la sem ambiguidade, isto é, pode saber qual o símbolo da fonte que foi codificado
 - O código do exemplo anterior é singular!
 - A condição de não singularidade é, em geral, insuficiente se se pretender usar o código para enviar, não um único símbolo, mas uma sequência de símbolos



Códigos Univocamente Descodificáveis

- Exemplo:
 - Considere-se:
 - $X = \{a, b, c, d\}$
 - $D = \{0, 1\}$
 - e o código binário C definido por:
 - $C(a) = 0, C(b) = 1, C(c) = 01, C(d) = 10$
 - Código claramente singular. Ao receber-se, por exemplo, a sequência:
 - 0110
 - não é possível determinar se a sequência de símbolos de fonte codificada foi:
 - cd, abba, abd, ou cba.
 - Para se evitar este tipo de ambiguidade, deve exigir-se ao código que seja “univocamente decodificável”



Códigos Univocamente Descodificáveis

- Códigos Univocamente Descodificáveis:
 - Seja $x_1, \dots, x_n$ uma sequência de n símbolos de X . Considere-se um código:
 - $C : X \rightarrow D$
 - A extensão de ordem n do código C , denotada C^n , é uma função de X^n para D^* definida pela simples concatenação das palavras de C , isto é:
 - $C^n(x_1, \dots, x_n) = C(x_1).C(x_2). \dots .C(x_n)$
- Exemplo:
 - A extensão de ordem 2 do código C definido no exemplo anterior é:
 - $C^2(aa) = C(a)C(a) = 00$ $C^2(ab) = C(a)C(b) = 01$ $C^2(ac) = C(a)C(c) = 001$
 - $C^2(ad) = C(a)C(d) = 010$ $C^2(ba) = C(b)C(a) = 10$ $C^2(bb) = C(b)C(b) = 11$
 - $C^2(bc) = C(b)C(c) = 101$ $C^2(bd) = C(b)C(d) = 110$ $C^2(ca) = C(c)C(a) = 010$
 - $C^2(cb) = C(c)C(b) = 011$ $C^2(cc) = C(c)C(c) = 0101$ $C^2(cd) = C(c)C(d) = 0110$
 - $C^2(da) = C(d)C(a) = 100$ $C^2(db) = C(d)C(b) = 101$ $C^2(dc) = C(d)C(c) = 1001$
 - $C^2(dd) = C(d)C(d) = 1010$



Códigos Univocamente Descodificáveis

- Códigos Univocamente Descodificáveis:
 - O código denotado C^* , a que se chama simplesmente extensão (sem ordem) do código C , é obtido do mesmo modo mas considerando todas as sequências de X de qualquer comprimento
 - Um código C é dito univocamente decodificável se a sua extensão C^* for não singular
 - Exemplo: A extensão C^* do código C definido anteriormente é:

$$\begin{aligned}C^*(a) &= C(a) = 0 \\C^*(b) &= C(b) = 1 \\C^*(c) &= C(c) = 01 \\C^*(d) &= C(d) = 10 \\C^*(aa) &= C(a)C(a) = 00 \\C^*(ab) &= C(a)C(b) = 01 \\&\vdots \\C^*(dc) &= C(d)C(c) = 1001 \\C^*(dd) &= C(d)C(d) = 1010 \\C^*(aaa) &= C(a)C(a)C(a) = 000 \\C^*(aab) &= C(a)C(a)C(b) = 001 \\&\vdots\end{aligned}$$



Códigos Instantâneos

- Códigos Univocamente Descodificáveis:
 - Assim, o código C não é univocamente decodificável porque:
 - $C^*(c) = 01$
 - $C^*(ab) = 01$
 - pelo que:
 - C^* é singular $\rightarrow$ corresponde a uma função não injetiva
- Códigos Instantâneos:
 - Embora a propriedade de decodificabilidade unívoca seja claramente desejável, pode ser, na prática, insuficiente:
 - para decodificar um símbolo, pode ser necessário observar muitos símbolos seguintes, o que dá ao processo de decodificação uma grande demora



Códigos Instantâneos

- Códigos Instantâneos (Exemplo):
 - Considere-se o código univocamente descodificável
 - $C : \{a, b, c, d\} \rightarrow \{0, 1\}$,
 - definido por $C(a) = 01$, $C(b) = 11$, $C(c) = 00$, $C(d) = 110$
 - Se um recetor receber, por exemplo, a sequência:

$$11 \underbrace{00 \dots 0 \dots 0}_{n \text{ zeros}} 11$$

- a sua descodificação é simples:

$$\underbrace{b c c \dots c \dots c b}_{\frac{n}{2} \text{ c's}} \Leftarrow n \text{ par}$$

$$\underbrace{d c c \dots c \dots c b}_{\frac{n-1}{2} \text{ c's}} \Leftarrow n \text{ ímpar.}$$



Códigos Instantâneos

- Códigos Instantâneos (exemplo – continuação):
 - Para decodificar o primeiro símbolo, pode ser necessário observar um número arbitrariamente grande de palavras de código subsequentes, introduzindo um grande atraso no processo de comunicação
 - Por este motivo, este código é dito não instantâneo
 - Nem sempre é possível identificar uma palavra de código de forma instantânea. É fácil constatar que este facto se deve a que uma das palavras do código:
 - $C(b) = 11$
 - é prefixo de uma outra palavra de código:
 - $C(d) = 110$
 - É evidente que, se nenhuma palavra de código for prefixo de outra, as palavras são decodificadas instantaneamente. Os códigos com esta propriedade são ditos instantâneos, ou de prefixo (*prefix codes*)



Desigualdade de *Kraft-McMillan*

- O objetivo de desenho de um código é obter o menor comprimento médio possível
- Não é possível reduzir arbitrariamente o comprimento de todas as palavras, mantendo o código instantâneo
- Exemplo:
 - num código binário, para um alfabeto de fonte com mais de 2 símbolos
 - se 0 for uma palavra de código
 - não pode existir mais nenhuma palavra com apenas um bit, pois todas as restantes têm de começar por 1 (para que 0 não seja prefixo de nenhuma)
- Este facto é expresso formalmente pelo teorema de *Kraft-McMillan*, mais conhecido como desigualdade de *Kraft-McMillan* (DKM)



Codificação de *Huffman*

- O algoritmo foi proposto por *Huffman* em 1952, para resolver o problema da codificação óptima de uma fonte sem memória
- Considera-se uma fonte X emitindo símbolos de um alfabeto
 - $X = \{x_1, \dots, x_N\}$, com probabilidades $\{p_1, \dots, p_N\}$
- O algoritmo de *Huffman* pode ser dividido em duas partes:
 - Parte A: Repetem-se os seguintes passos:
 - Passo A.1: Ordenam-se os símbolos por ordem decrescente de probabilidade
 - Passo A.2: Agrupam-se os 2 símbolos menos prováveis num “super-símbolo” cuja probabilidade é a soma das probabilidades dos dois símbolos agrupados (o alfabeto resultante possui um símbolo a menos)
 - Passo A.3: Se o alfabeto resultante possui dois ou mais símbolos, volta-se ao Passo A.1; caso contrário, está concluída a parte A



Codificação de *Huffman*

- O algoritmo de *Huffman* (continuação):
 - Parte B:
 - A Parte A do algoritmo produziu uma árvore binária na qual as folhas (nós sem descendentes) correspondem aos símbolos da fonte
 - Basta agora percorrer a árvore da raiz até cada uma das folhas, atribuindo (de forma arbitrária) os símbolos “0” e “1” a cada par de ramos com origem em cada nó interno (não folha) da árvore
 - No caso de existirem, em algum ponto do algoritmo, dois ou mais símbolos (ou “super-símbolos”) com o mesmo valor de probabilidade, a ordenação destes símbolos é arbitrária, sem que isso tenha qualquer impacto no comprimento médio do código resultante
- Exemplo:
 - Considere-se o alfabeto
 - $X = \{a, b, c, d\}$
 - com probabilidades, respetivamente:
 - $\{0.1, 0.25, 0.2, 0.45\}$



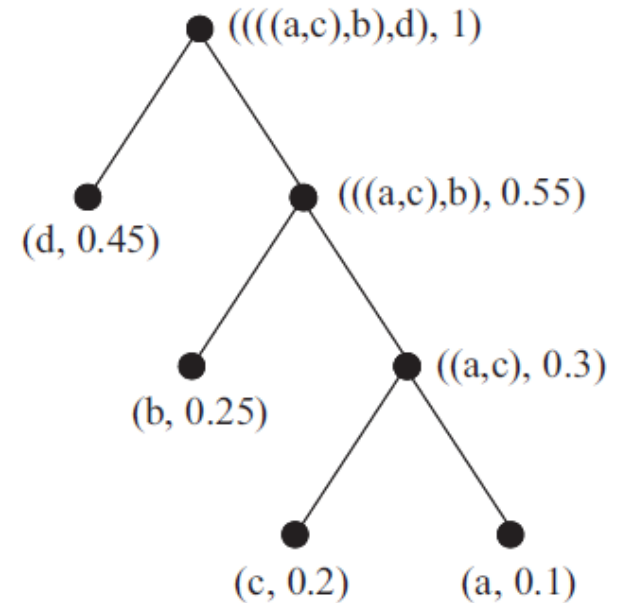
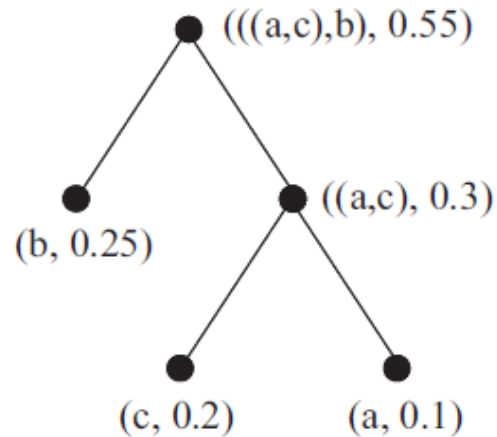
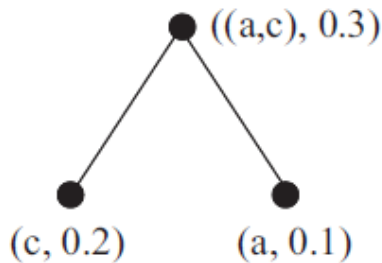
Codificação de *Huffman*

- Resolução:
 - O primeiro passo consiste em ordenar o alfabeto por ordem decrescente de probabilidades
 - o resultado desta ordenação é: {d, b, c, a}
 - com probabilidades ordenadas {0.45, 0.25, 0.2, 0.1}
 - No passo A.1 agrupam-se os dois símbolos menos prováveis, c e a, num “super-símbolo” (a, c) com probabilidade $p(a) + p(c) = 0.1 + 0.2 = 0.3$
 - Este passo está ilustrado na figura do slide seguinte
 - Junto a cada nó da árvore binária que vai sendo construída está indicado qual o conjunto de símbolos que lhe corresponde e qual a correspondente probabilidade total
 - O alfabeto resultante é: {d, b, (a, c)}, com probabilidades {0.45, 0.25, 0.3}
 - Os símbolos a e c do alfabeto original foram substituídos pelo “super-símbolo” (a, c)



Codificação de *Huffman*

- Sequência de árvores binárias produzidas pela parte A do algoritmo de *Huffman*:



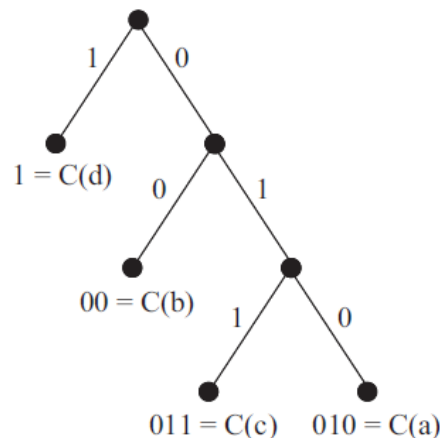
Codificação de *Huffman*

- Resolução:
 - Dado que este alfabeto possui dois ou mais (neste caso três) símbolos, volta-se ao passo A.1
 - Reordenando o alfabeto obtém-se $\{d, (a, c), b\}$, com probabilidades ordenadas $\{0.45, 0.3, 0.25\}$
 - Procede-se agora ao passo A.2, no qual se agrupam os símbolos de menor probabilidade: (a, c) e b , num novo “super-símbolo” $((a, c), b)$
 - O alfabeto resultante é $\{d, ((a, c), b)\}$, com probabilidades $\{0.45, 0.55\}$
 - Reordenando e agrupando, obtém-se finalmente um alfabeto com apenas um símbolo $\{((a, c), b, d)\}$, naturalmente com probabilidade 1
 - A árvore final que se obtém tem a raiz corresponde ao “super-símbolo” final $((a, c), b, d)$



Codificação de *Huffman*

- Resolução:
 - Na parte B do algoritmo, toma-se a árvore produzida na parte A e etiquetam-se os ramos que emanam de cada bifurcação, nos caminhos que vão da raiz até às folhas, com o símbolo “1” para um dos ramos e o símbolo “0” para o outro
 - Para se obter a palavra de código para cada símbolo, basta registrar as etiquetas binárias dos ramos percorridos no caminho da raiz até esse símbolo. Este procedimento está representado na figura abaixo



Codificação de *Shannon-Fano-Elias*

- A codificação designada de *Shannon-Fano-Elias* (SFE) é um híbrido de várias propostas apresentadas separadamente por *Shannon*, *Fano* e *Elias* nas décadas de 1940 e 1950
- Como foi visto, para que o comprimento médio de codificação se aproxime do limite teórico inferior imposto pela entropia da fonte, pode ser necessário recorrer a extensões de ordem elevada. Esta opção, no entanto, pode tornar-se pouco prática pelo seguinte motivo:
 - O alfabeto para uma extensão de ordem n de uma fonte com um alfabeto original de N símbolos possui N^n símbolos
 - Se se adoptar codificação de *Huffman* para os símbolos extendidos, é necessário desenhar um código de *Huffman* para um alfabeto com N^n símbolos



Codificação de *Shannon-Fano-Elias*

- Exemplo:
 - Se o alfabeto original for constituído pelos 256 símbolos ASCII, o alfabeto da extensão de ordem 3 possui mais de 16 milhões de símbolos
 - $256^3 = 2^{24} = 16\,777\,216$
 - Seria pois necessário desenhar um código de *Huffman* para um alfabeto com este enorme número de símbolos, a maioria dos quais acabariam mesmo por nunca ser utilizados
- A característica fundamental da codificação de SFE é a possibilidade de se obter a palavra de código para um único símbolo, sem necessidade de criar palavras de código para todos os símbolos do alfabeto
 - É claro que a codificação de *Huffman* não possui esta característica



Codificação de *Shannon-Fano-Elias*

- Considere-se um alfabeto $X = \{x_1, \dots, x_N\}$, cujos símbolos são emitidos com probabilidades $\{p_1, \dots, p_N\}$
- Assume-se que $p_i > 0$, para $i = 1, \dots, N$; se existir algum $p_i = 0$, pode simplesmente retirar-se o respectivo símbolo do alfabeto pois nunca vai ser necessário codificá-lo
- No caso dos códigos binários, $C: X \rightarrow D = \{0, 1\}$
 - A generalização para outros alfabetos de código é trivial
- Considere-se a função de distribuição cumulativa $F(x)$ que se define como:

$$F(x_i) = F_i = \sum_{j=1}^i p_j, \quad \text{para } i = 1, 2, \dots, N$$

- E $F_0 = 0$



Codificação de *Shannon-Fano-Elias*

- Dado que todos os p_i são estritamente positivos, a sequência:
 - $F_0, F_1, F_2, \dots, F_N$
- É estritamente monótona, isto é:
 - $F_0 < F_1 < F_2 < \dots < F_N$
 - Note-se ainda que $F_1 = p_1$ e $F_N = 1$
- Dado que a sequência $F_1, F_2, \dots, F_N$ é estritamente monótona, todos os F_i são diferentes, isto é:
 - $(i \neq j) \Rightarrow (F_i \neq F_j)$
 - Definição de código não singular
- Esta observação sugere que se utilizem os números $F_1, F_2, \dots, F_N$ para codificar os símbolos $\{x_1, \dots, x_N\}$

Codificação de *Shannon-Fano-Elias*

- **Problema:** em geral, os números $F_1, F_2, \dots, F_N$ são reais arbitrários, no intervalo $]0, 1]$
 - a sua escrita (em base 2, ou em qualquer outra base) pode exigir um número infinito de dígitos
- Para se obter um código útil, é necessário truncar as representações dos números $F_1, F_2, \dots, F_N$ para comprimentos finitos
- Ao fazer essa truncatura, pode exigir-se que o código obtido, mais do que simplesmente não singular, seja instantâneo
- No procedimento de SFE não se utilizam os números $F_1, F_2, \dots, F_N$, mas sim um outro conjunto de números, com estes relacionados



Codificação de *Shannon-Fano-Elias*

- Considera-se que a cada símbolo x_i se faz corresponder o intervalo
 - $[F_{i-1}, F_i[$ -> fechado à esquerda e aberto à direita
- cuja largura é p_i
 - $F_i - F_{i-1} = p_i$
- Considerem-se agora os pontos centrais de cada um destes intervalos, que serão designados como $\bar{F}_i$, para $i = 1, 2, \dots, N$
- Dado que a largura do i -ésimo intervalo é p_i , tem-se que:

$$\bar{F}_i = F_{i-1} + \frac{p_i}{2} = F_i - p_i + \frac{p_i}{2} = F_i - \frac{p_i}{2}$$



Codificação de *Shannon-Fano-Elias*

- Dado que todos os p_i são estritamente positivos:

$$\begin{aligned}\overline{F}_{i+1} - \overline{F}_i &= F_i + \frac{p_{i+1}}{2} - F_{i-1} - \frac{p_i}{2} \\ &= \underbrace{F_i - F_{i-1}}_{p_i} - \frac{p_i}{2} + \frac{p_{i+1}}{2} \\ &= \frac{p_i + p_{i+1}}{2} > 0\end{aligned}$$

- Assim, a sequência $\overline{F}_1, \overline{F}_2, \dots, \overline{F}_N$ também é estritamente monótona e, como tal, também verifica a propriedade de não singularidade, podendo ser usada para construir um código para os símbolos $\{x_1, \dots, x_N\}$
- Resta encontrar uma forma de truncar as representações dos números $F_1, F_2, \dots, F_N$ que não destrua a propriedade de não singularidade e que, também, produza um código instantâneo



Codificação de *Shannon-Fano-Elias*

- $l_i = \lceil -\log_2 p_i \rceil + 1 \rightarrow$ comprimento da palavra de código $C(x_i)$
- As palavras de código são dadas simplesmente por:
 - $C(x_i) =$ primeiros l_i dígitos binários de $\bar{F}_i$
- Todos os números $\bar{F}_i$ são menores que 1
- Todos possuem escritas da forma $\bar{F}_i = 0.d_1 d_2 \dots$
 - possivelmente com um número infinito de dígitos
- Na palavra de código não se inclui o zero à esquerda do ponto, pelo que:
 - $C(x_i) = d_1 d_2 \dots d_{l_i}$



Codificação de *Shannon-Fano-Elias*

- **Exemplo (1) - Codificação de SFE decimal:**
- Considere-se uma fonte que gera símbolos do alfabeto:
 - $X = \{x_1, x_2, \dots, x_{10}, x_{11}\}$
- com probabilidades:
 - $\{1/2, 1/4, 1/8, 1/16, 1/32, 1/64, 1/128, 1/256, 1/512, 1/1024, 1/1024\}$
- Construa uma tabela para cada palavra de código com os valores:
 - $i, p_i, -\log_{10} p_i, l_i, \bar{F}_i, C(x_i)$

i	p_i	$-\log_{10} p_i$	l_i	$\bar{F}_i$	$C(x_i)$
1	0.500000000000	0.3010	2	0.250000000000	25
2	0.250000000000	0.6020	2	0.625000000000	62
3	0.125000000000	0.9030	2	0.812500000000	81
4	0.062500000000	1.2041	3	0.906250000000	906
5	0.031250000000	1.5051	3	0.953125000000	953
6	0.015625000000	1.8061	3	0.976562500000	976
7	0.007812500000	2.1072	4	0.988281250000	9882
8	0.003906250000	2.4082	4	0.994140625000	9941
9	0.001953125000	2.7092	4	0.997070312500	9970
10	0.000976562500	3.0102	5	0.998535156250	99853
11	0.000976562500	3.0102	5	0.999511718750	99951



Codificação de *Shannon-Fano-Elias*

- O comprimento médio deste código é de 2.14257 dígitos decimais por símbolo, enquanto que a entropia de base 10 da fonte é de 0.60147 dígitos decimais por símbolo
- **Exemplo (2) - Codificação de SFE binária:**
- Repita o exemplo anterior, agora para codificação de SFE binária

i	p_i	$-\log_2 p_i$	l_i	$\bar{F}_i$	$(\bar{F}_i)_{(2)}$	$C(x_i)$
1	0.5000000000	1	2	0.2500000000	0.010000000000	01
2	0.2500000000	2	3	0.6250000000	0.101000000000	101
3	0.1250000000	3	4	0.8125000000	0.110100000000	1101
4	0.0625000000	4	5	0.9062500000	0.111010000000	11101
5	0.0312500000	5	6	0.9531250000	0.111101000000	111101
6	0.0156250000	6	7	0.9765625000	0.111110100000	1111101
7	0.0078125000	7	8	0.9882812500	0.111111010000	11111101
8	0.0039062500	8	9	0.9941406250	0.111111101000	111111101
9	0.0019531250	9	10	0.9970703125	0.111111110100	1111111101
10	0.0009765625	10	11	0.99853515625	0.111111111010	11111111101
11	0.0009765625	10	11	0.99951171875	0.111111111100	11111111111

- O comprimento médio deste código é de 2.9980 bits por símbolo, enquanto que a entropia binária da fonte é de 1.9980 bits por símbolo



Bibliografia

- *Mário A. T. Figueiredo, “Elementos de Teoria da Informação Versão 1.5”, Departamento de Engenharia Electrotécnica e de Computadores, Instituto Superior Técnico de Lisboa, 2007.*
- *Simon Haykin, Communication Systems, McMaster Univ, 5th Edition, 2009 (ISBN 978-047-16979-0-9).*
- *A. Bruce Carlson, Paul Crilly. Communication Systems – An Introduction to Signals and Noise, McGraw-Hill, 5th edition, 2009 (ISBN 978-007-338-040-7).*



Bibliografia

